CSC 710 Software Engineering

Instructor: Dr. Tianxiao Zhang

Software Construction

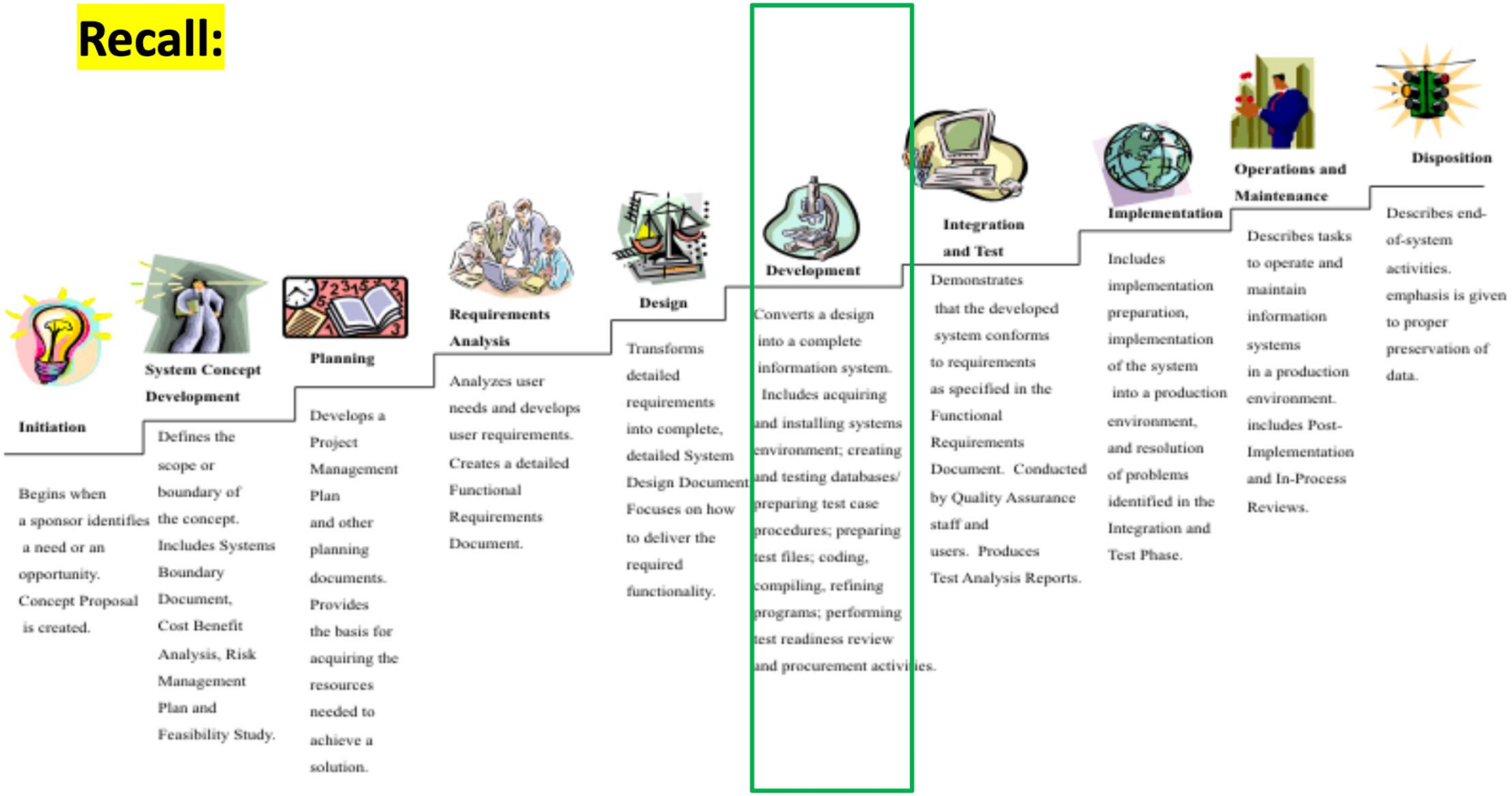
Recall:

- SWEBOK V3.0 (2014) is the most recent version of the IEEE Guide to the Software Engineering Body of Knowledge.
- According to SWEBOK Version 3:
  - Software construction is a software engineering discipline.
  - It is the detailed creation of working meaningful software through a combination of coding, verification, unit testing, integration testing, and debugging.
  - It is linked to all the other software engineering disciplines, most strongly to software design and software testing.
- ACM separates these activities differently
  - Software Construction: coding and debugging
  - Software Verification and Validation: verification, unit testing, and integration testing

Software Development Phases

Recall:

Software Construction



Software Construction Topics

- Greenfield & Brownfield
- Commenting
- Design by Contract (DbC)

Greenfield & Brownfield

- Greenfield Development happens when you start a brand new project, as in, clean slate development.

- Brownfield Development happens when a business decides to improve upon an existing application infrastructure.

Commenting

Comments help Brownfield programmers

And remember you will more than likely be a “Brownfield Programmer”

Comments also help <u>you</u> remember why you coded something the way you did.

Design by Contract (DbC)

- Design by contract (DbC), also known as contract programming, programming by contract, and design-by-contract programming, is an approach for designing software.

- For Design by Contract (DbC), methods and components must follow specific rules to ensure the software works correctly. These rules are like agreements, called contracts.

Design by Contract (DbC)

Main Contracts:

- **Preconditions**: What must be true before a method runs.
- **Postconditions**: What must be true after the method runs if it started correctly.
- **Invariants**: Conditions that must always be true for an object or system.

Software Construction Topics

- Greenfield & Brownfield
- Commenting
- Design by Contract (DbC)
- **Open-Source Software Components**
- Software Re-engineering
- Software Maintenance
- Refactoring

Open-Source Software Statistics

- 95% of enterprises use open-source software.
- Open-source software powers 99% of all supercomputers worldwide.
- More than 70% of software developers use open-source code in their projects.
- Open-source software contributes to over $60 billion in savings for businesses annually.
- The open-source community has over 15 million developers globally.
- Open-source software adoption has increased by 44% since 2023.
- The majority of IoT devices run on open-source software.

Open-Source Software Statistics

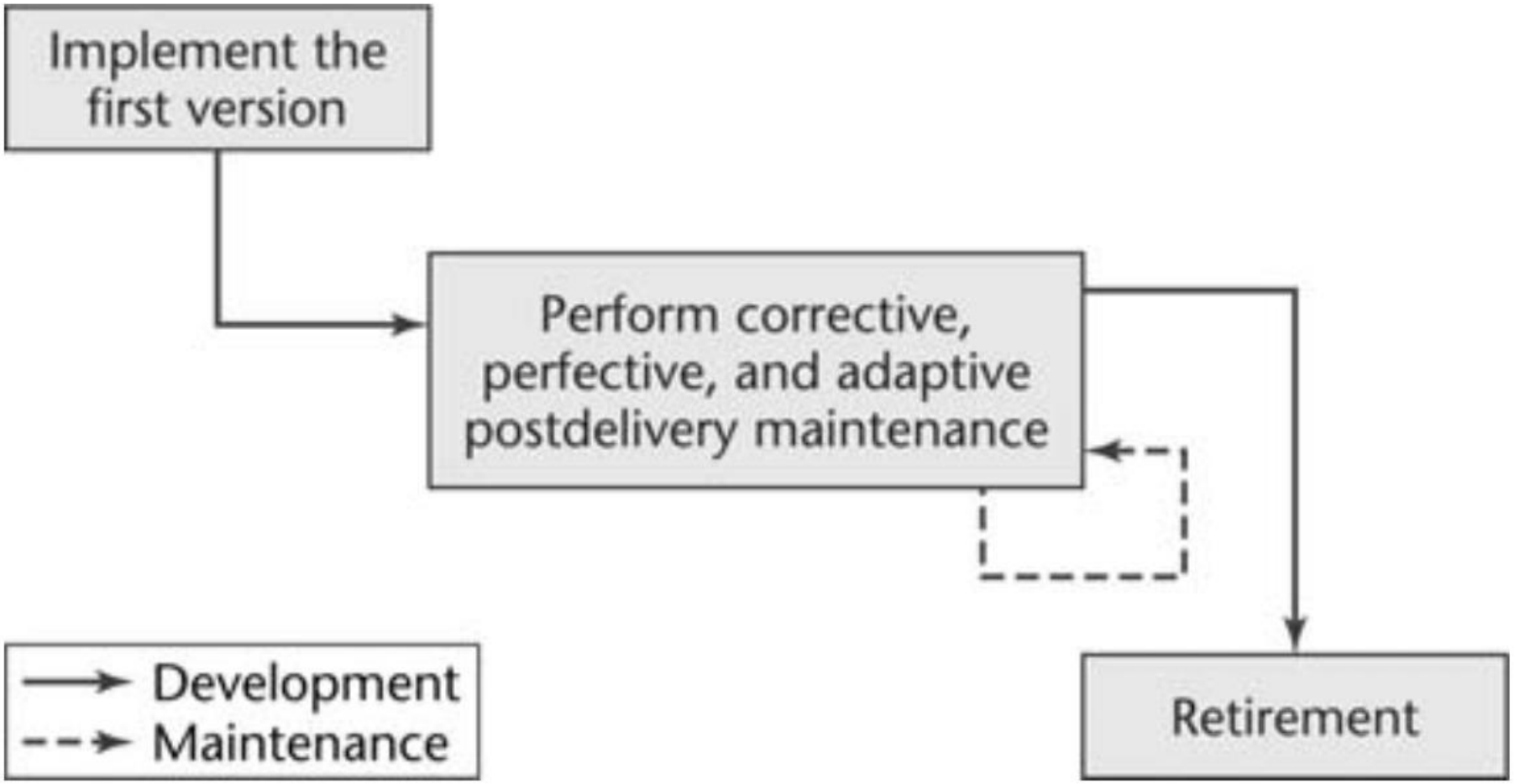
- Over 80% of smartphones run on open-source software, predominantly android.
- Open-source software reduces software development costs by 20-30%.
- Open-source projects contain an average of 249 unique contributors.
- 57% of companies have an open-source software program or plan to establish one.
- Open-source drives digital transformation in 78% of organizations.
- 88% of companies say open-source improves their security posture.
- Open-source software contributes to $166 billion in R&D savings annually.
- The average lifespan of an open-source project is 2.2 years.

Open-Source Life-Cycle Model

- Open-source software is a type of computer software in which source code is released under a license in which the copyright holder grants users the rights to use, study, change, and distribute the software to anyone and for any purpose.

- Open-source software is developed using the Open-Source Life-Cycle Model.

Open-Source Life-Cycle Model



Open-Source Life-Cycle Model

Extremely successful for certain infrastructure projects

- Languages: Python
- Operating system: Linux, OpenBSD, Mach, Darwin
- Web browsers: Firefox, Netscape and Web servers: Apache
- Compilers: gcc
- Database management systems: MySQL

Open-Source Advantages

- Shift from low-value work to high-value work
  - Don’t spend time reinventing the wheel
  - Use the world’s best wheels
  - Stand on the shoulders of technology giants
- Lower cost
  - Open-Source software often comes at no cost or relatively low cost
  - Additional costs like training, maintenance, and support are sunk costs.
  - You're going to be paying for both whether the software is open or closed source.

Open-Source Advantages

- Faster to market
  - You'd be hard pressed to find a startup today worth it's venture capital funding not based, at least in part (if not largely) on open-source.
- Better quality
  - More eyes looking for bugs
  - More chances for bugs to show up because it is used in more systems
- More robust
  - Building software for a single use case vs. multiple use cases
  - Open-source, by its nature, is built for a variety of use cases, environments, and users.

Using Open-Source Components

Do Not Copy & Paste

- When you copy a code snippet, you also copy its bugs and vulnerabilities, but you lose visibility.
- Once you copy and paste a few lines you have no way of tracking this snippet and update it when a software bug or a security vulnerability is discovered and fixed with a patch or a new version.

Using Open-Source Components

Manage Changes with Adapter/Wrapper/Translator Design Pattern

- Recall a Wrapper converts the interface of a class into another interface clients expect.
- A Wrapper lets classes work together that could not otherwise because of incompatible interfaces.
- Writing a Wrapper will allow you to easily replace the library with another, should the need arise.
- It will also centralize the usage and will help you to monitor the places where the Wrapper is used.
- The Wrapper also helps with encapsulation and information hiding (Design Principle), making sure that any functionality you don't want to use is "hidden".

Using Open-Source Components

Forking or Cloning

- The right to "fork" or clone an open-source project and take the copy in a new direction is one of open-source's merits.
- If you do not wish to download an open-source library and use it ‘as-is’, because you’d like to modify it, then your best practice should be to fork or clone it.
- Forking (or cloning it) will enable you to track improvements made to this open-source component since you will always have a link to the original repository.
- The disadvantage here might be the need to compile and build your own package.

Using Open-Source Components

Keep Track to Know when You Need to Update

- A recent survey pointed out that 1 out of 16 open-source components contains a security vulnerability and many other libraries contain critical severe software bugs.
- The majority of issues (both security vulnerabilities and software bugs) are fixed in a newer version.
- Unless you are tracking your inventory and the updates of the libraries you are using …
- You won’t notice these problems until a hacker breaches your application.

Open-Source Component Risks

- It comes with some risk in three main areas:
  - Security
  - Open-Source Software Licensing Compliance
  - Quality

Open-Source Security Risks

Security

- Open-source security vulnerabilities are an extremely lucrative opportunity for hackers.
- Once discovered by the security research community, open-source vulnerabilities and the details on how to carry out the exploit are made public to everyone.
- This provides hackers with all the information that they need in order to carry out an attack.

Open-Source Security Risks

Security

- To make matters worse, since open-source usage is so widespread, a vulnerability in a popular open-source component provides hackers with many potential exploit victims.
- Today, known open-source vulnerabilities are published across a variety of security advisories and databases, and not in one centralized location.
- Tracking open-source software security vulnerabilities and their fixes requires an organization to employ specific tools and processes.

Open-Source License Compliance Risks

Open-Source Software Licensing Compliance

- Every open-source software component, along with its dependencies, comes with a license.
- When we use an open-source component in our project, we are agreeing to a set of terms and conditions that we must comply with.
- There are over 200 types of open-source licenses out there, each with its special, specific, and often confusing terms and conditions.
- Not tracking open-source licenses can leave an organization open to a lawsuit which can lead to relinquishing the exclusive ownership of the proprietary code.

Open-Source Quality Risks

Quality

- An organization invests many resources in the quality assurance of its proprietary code, but may overlook checking an open-source component's quality.
- How can we ensure that the open-source components that we are using won't risk the project's quality?
- At WhiteSource, the quality of an open-source component is rated based on three aspects of an open-source component:
  - Number of commits, as an indicator of its level of activity.
  - How many bugs are fixed in each specific version.
  - The amount and severity of open bugs for each specific version.

Software Construction Topics

- Greenfield & Brownfield
- Commenting
- Design by Contract (DbC)
- Open-Source Software Components
- **Software Re-engineering**
- Software Maintenance
- Refactoring

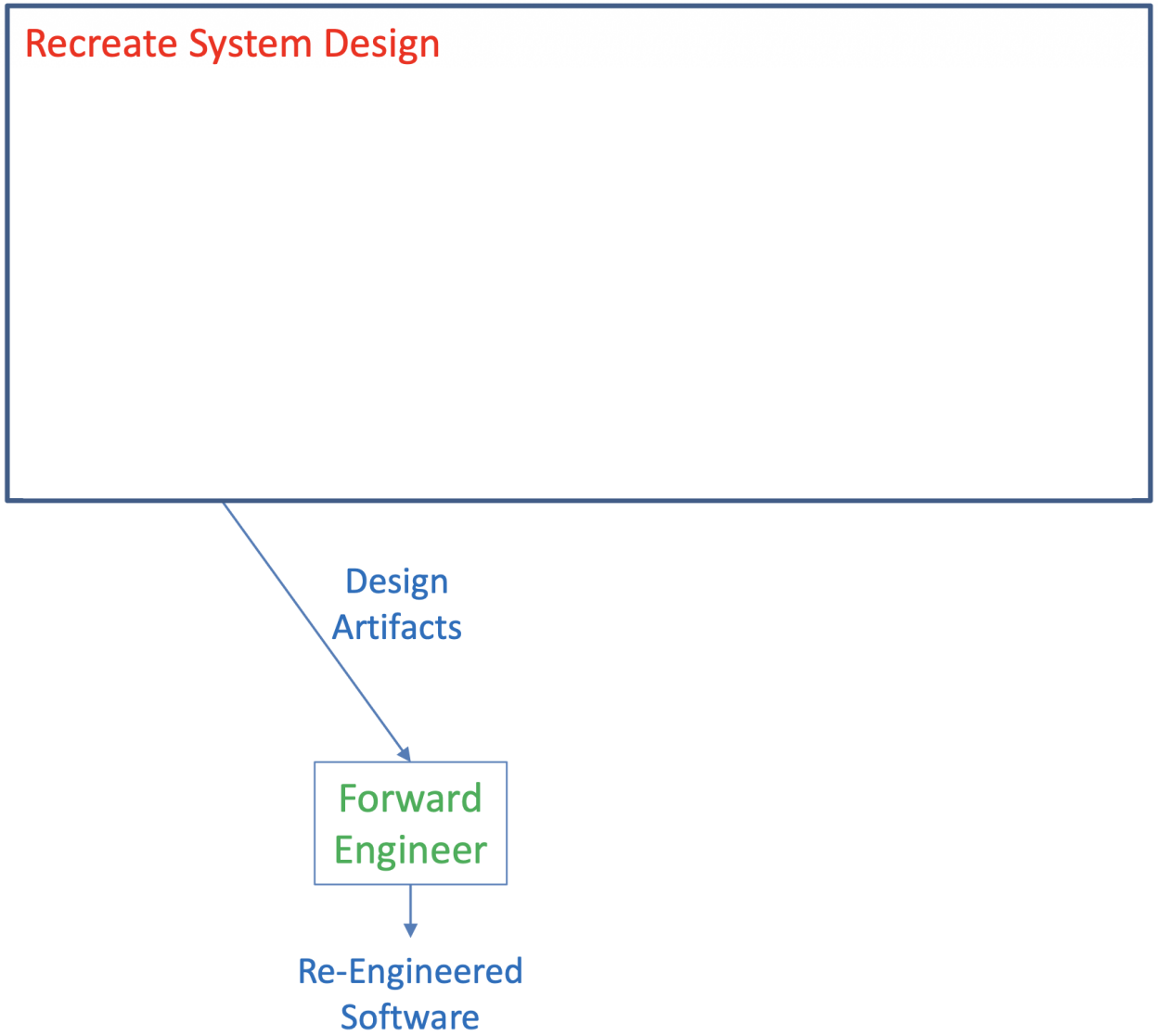
Software Re-Engineering

- Re-engineering is the examination and alteration of a system to reconstitute it in a new form.
- Why would you re-engineer software?
  - No source code and the processor the executable runs on is being retired
  - No source code and functionality is not what it needs to be to remain competitive in market
  - No source code and new federal guideline requires changes
  - Have source code, but in a language that no one is trained in
  - Have source code, but programmers familiar with code retired or left company
  - Have source code, but no compiler available for new hardware
  - Have source code, but cost of maintenance (all kinds) is too high and management believes a newer language would reduce maintenance costs (e.g., assembly language vs. Python or C++)

Forward Engineering

Software re-engineering consists of two steps:

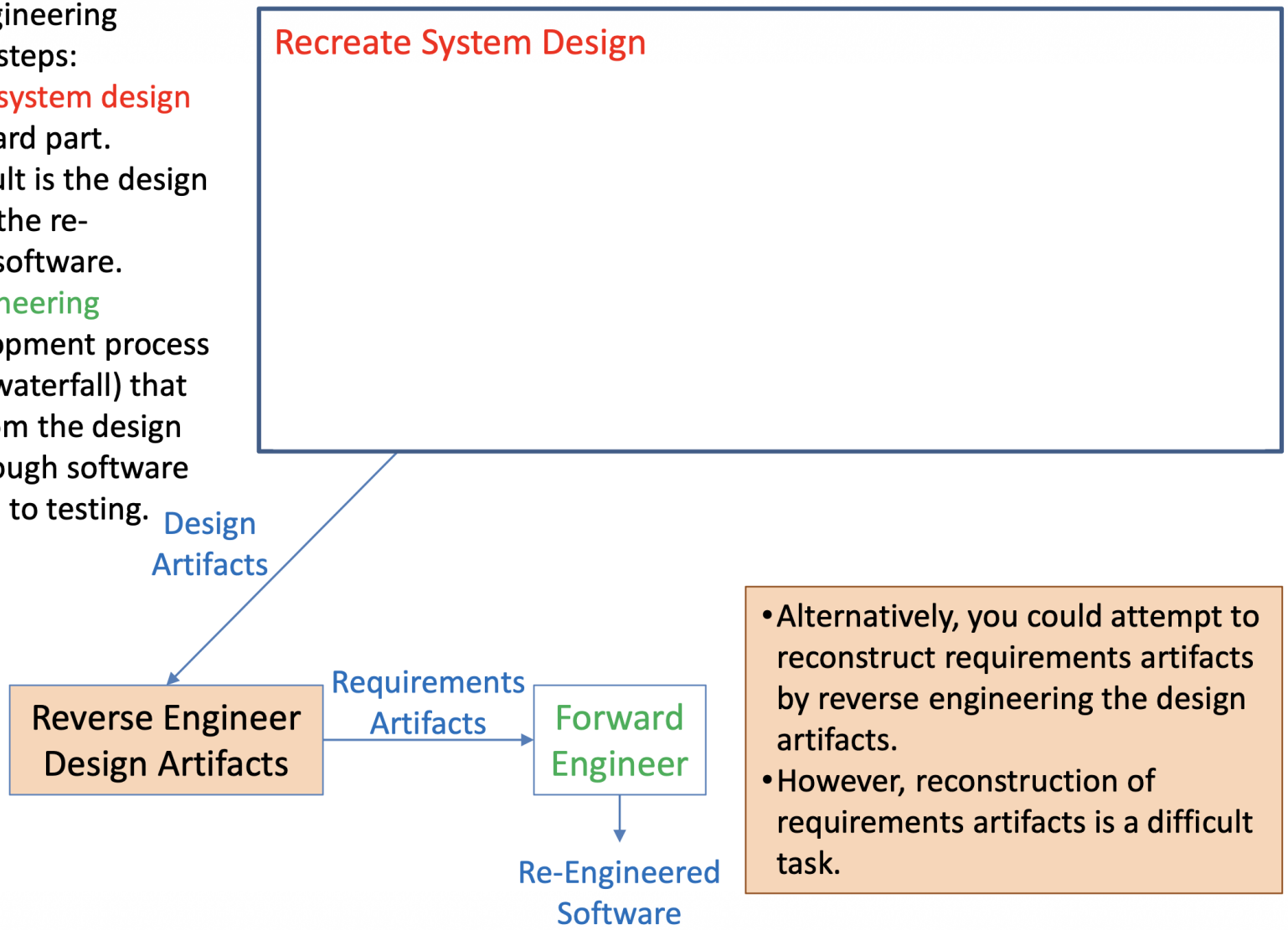
1. Recreate the system design
   This is the hard part.
   The end result is the design artifacts for the re-engineered software.
2. Forward engineering
   Usual development process (e.g., Agile, waterfall) that proceeds from the design artifacts through software construction to testing.



Reverse Engineer Design Artifacts

Software re-engineering consists of two steps:

1. Recreate the system design
   This is the hard part.
   The end result is the design artifacts for the re-engineered software.
2. Forward engineering
   Usual development process (e.g., Agile, waterfall) that proceeds from the design artifacts through software construction to testing.

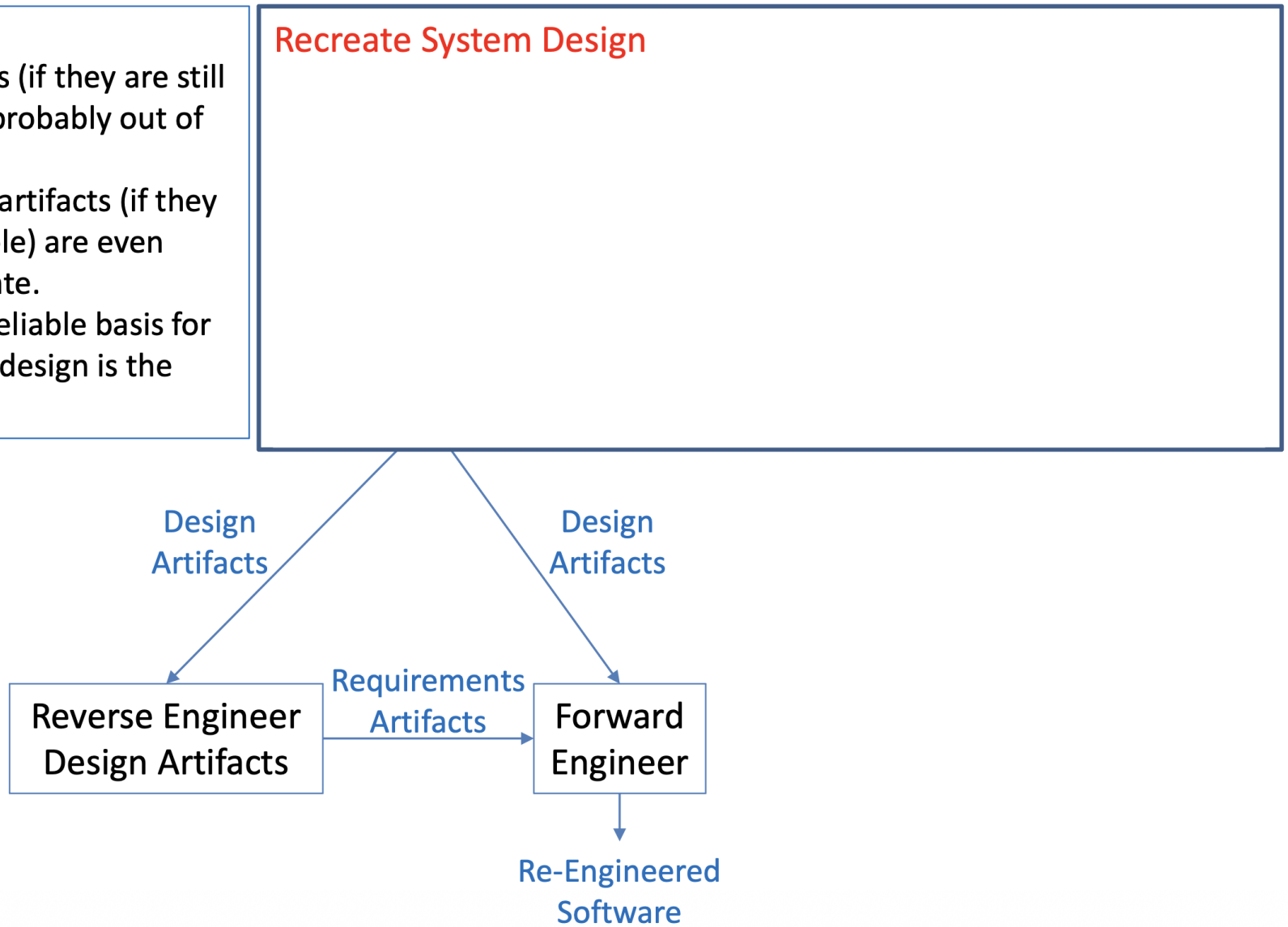


- Alternatively, you could attempt to reconstruct requirements artifacts by reverse engineering the design artifacts.
- However, reconstruction of requirements artifacts is a difficult task.

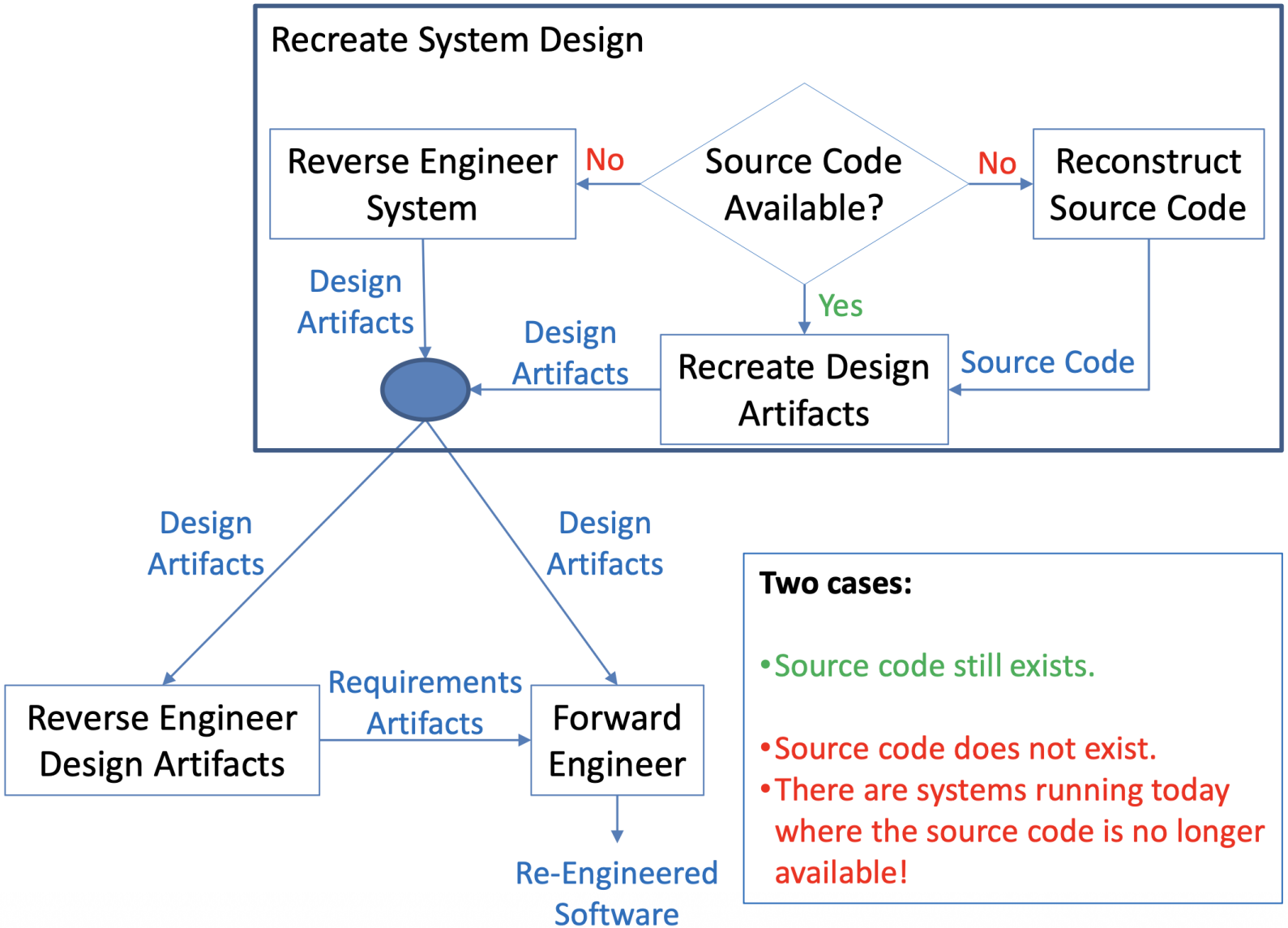
Recreate the System Design

Remember:

- Design artifacts (if they are still available) are probably out of date.
- Requirements artifacts (if they are still available) are even more out of date.
- So, your only reliable basis for recreating the design is the source code.



Recreate the System Design



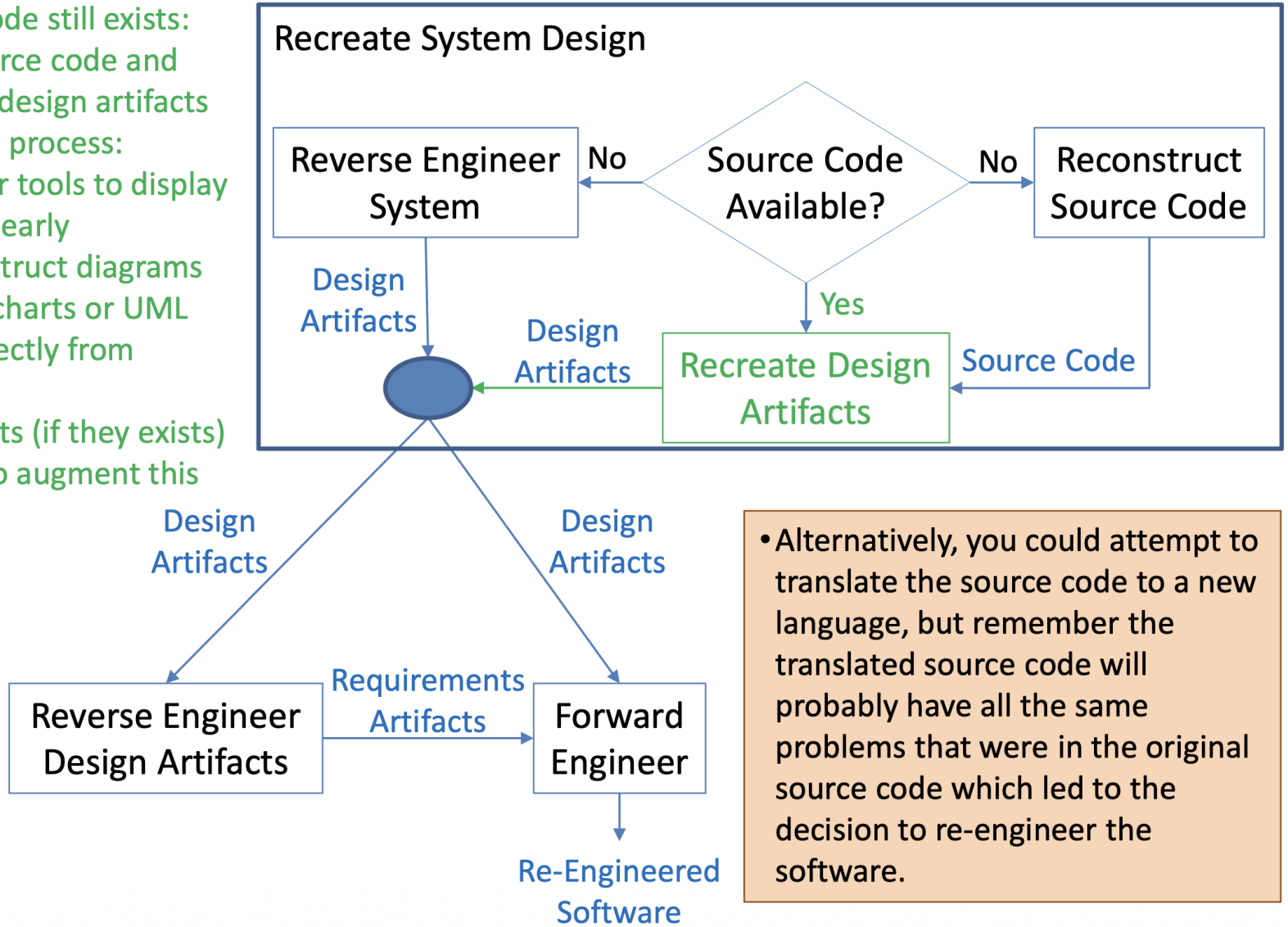
Two cases:

- Source code still exists.
- Source code does not exist.
- There are systems running today where the source code is no longer available!

Recreate the System Design from Source Code

If the source code still exists:

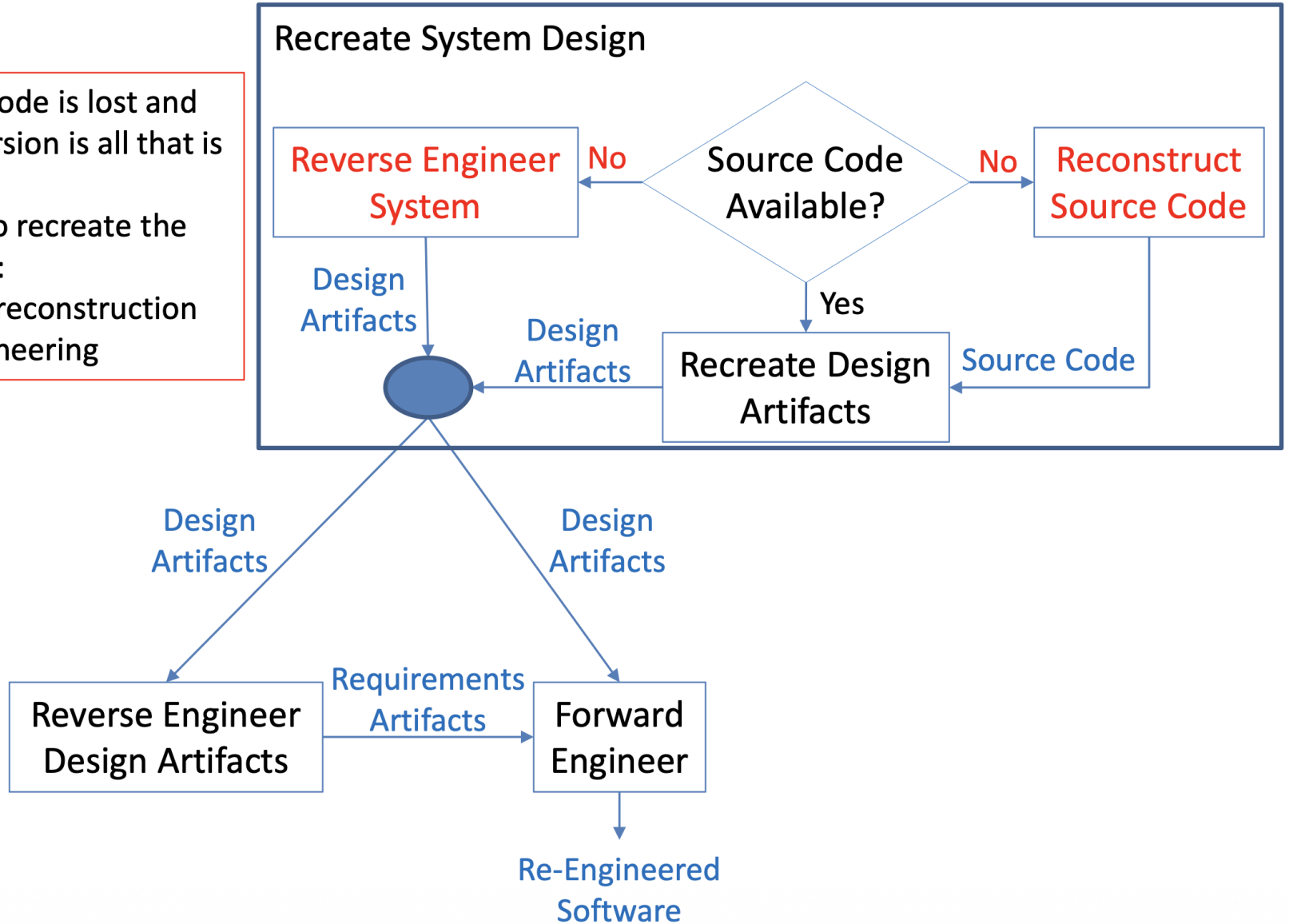
- Start with source code and recreate new design artifacts
- CASE tools for process:
  - Pretty printer tools to display code more clearly
  - Tools to construct diagrams such as flowcharts or UML diagrams directly from source code
- Testing artifacts (if they exists) can be used to augment this process



- Alternatively, you could attempt to translate the source code to a new language, but remember the translated source code will probably have all the same problems that were in the original source code which led to the decision to re-engineer the software.

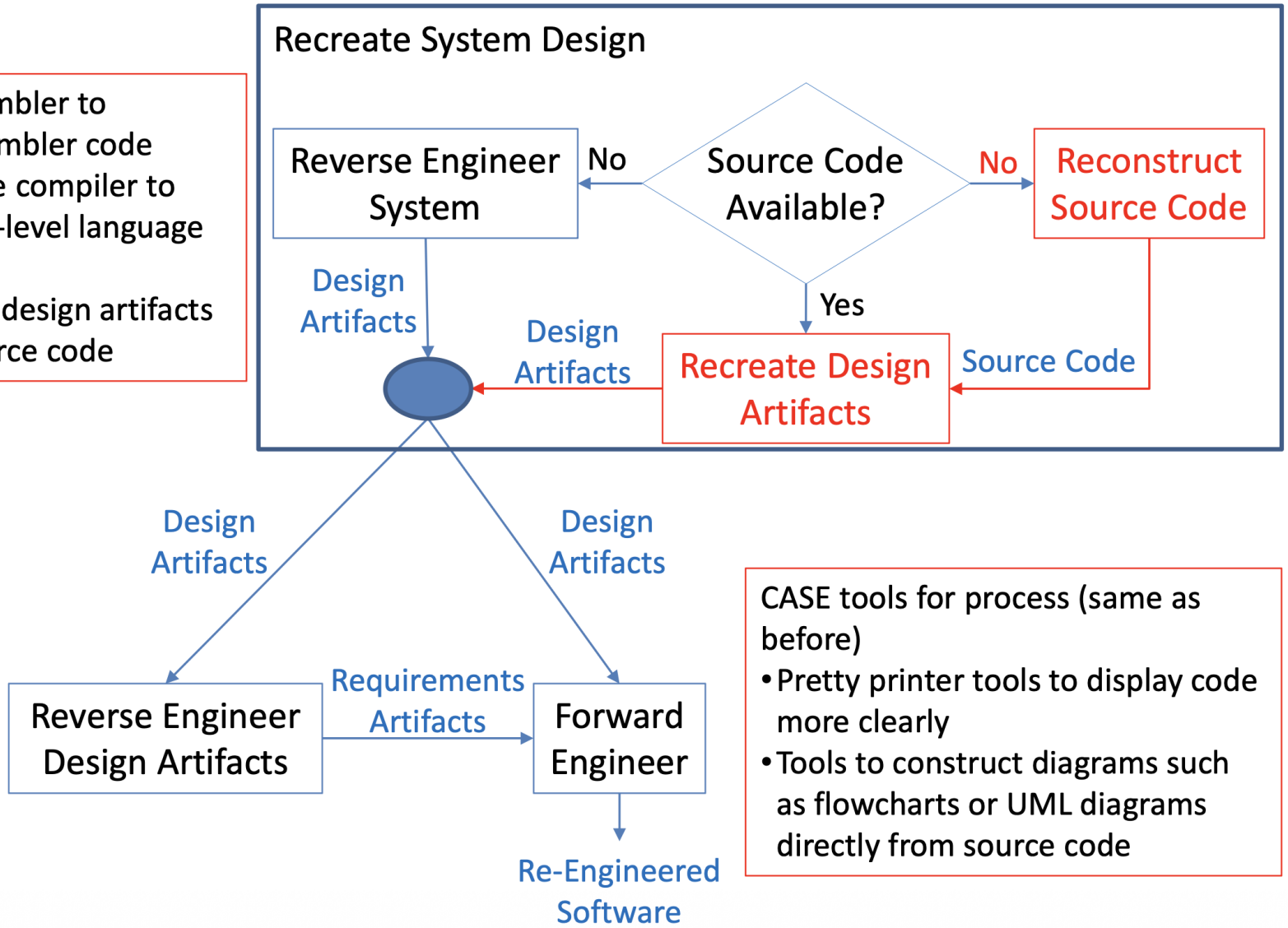
Recreate the System Design with No Source Code

- If the source code is lost and executable version is all that is available:
- Two options to recreate the system design:
  - Source code reconstruction
  - Reverse engineering



Reconstruct Source Code

- Use a disassembler to generate assembler code
- Then a reverse compiler to generate high-level language source code
- Recreate new design artifacts from new source code



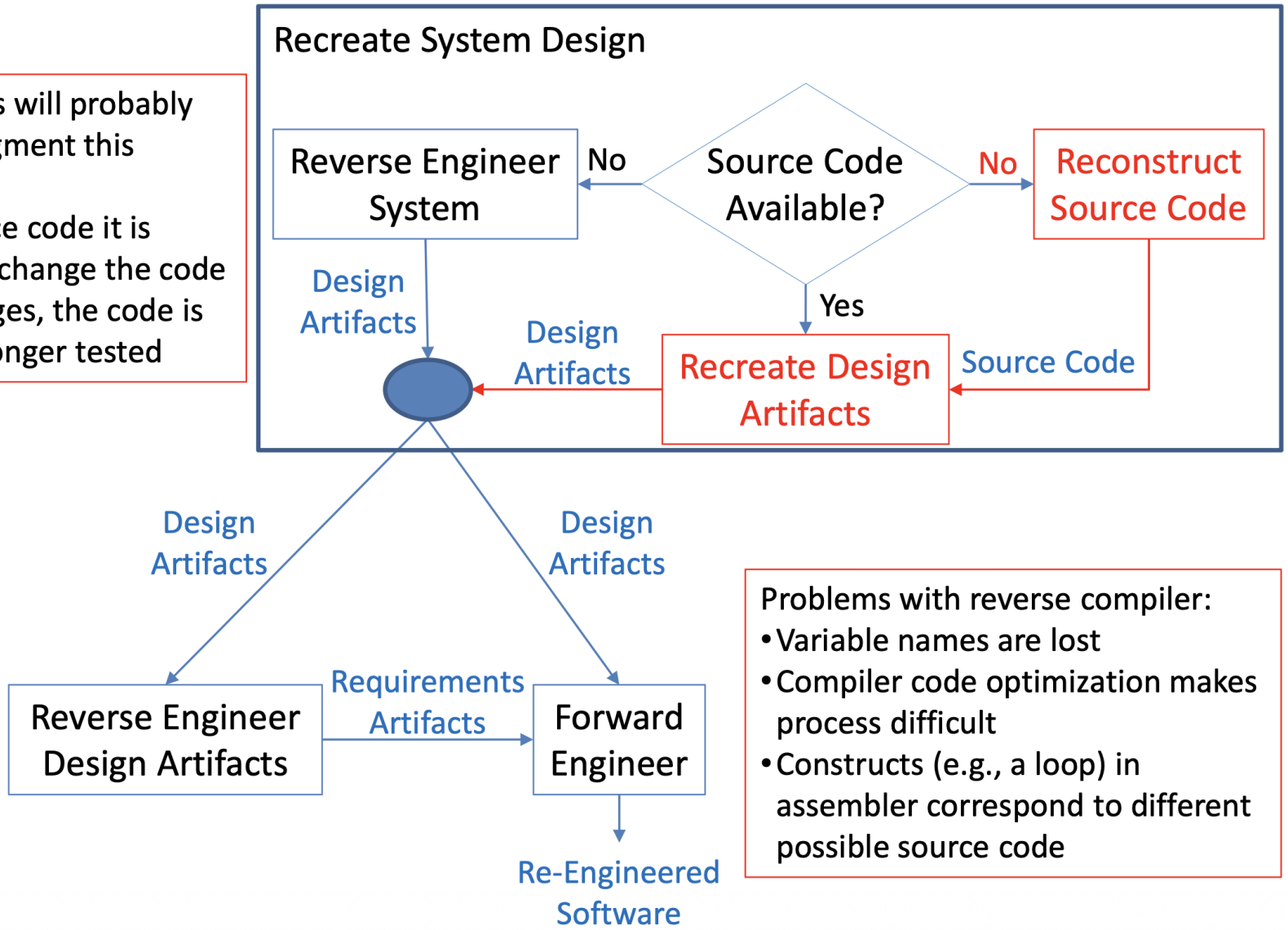
CASE tools for process (same as before)

- Pretty printer tools to display code more clearly
- Tools to construct diagrams such as flowcharts or UML diagrams directly from source code

Reconstruct Source Code

Testing artifacts will probably not exist to augment this process

- Without source code it is impossible to change the code
- With no changes, the code is probably no longer tested

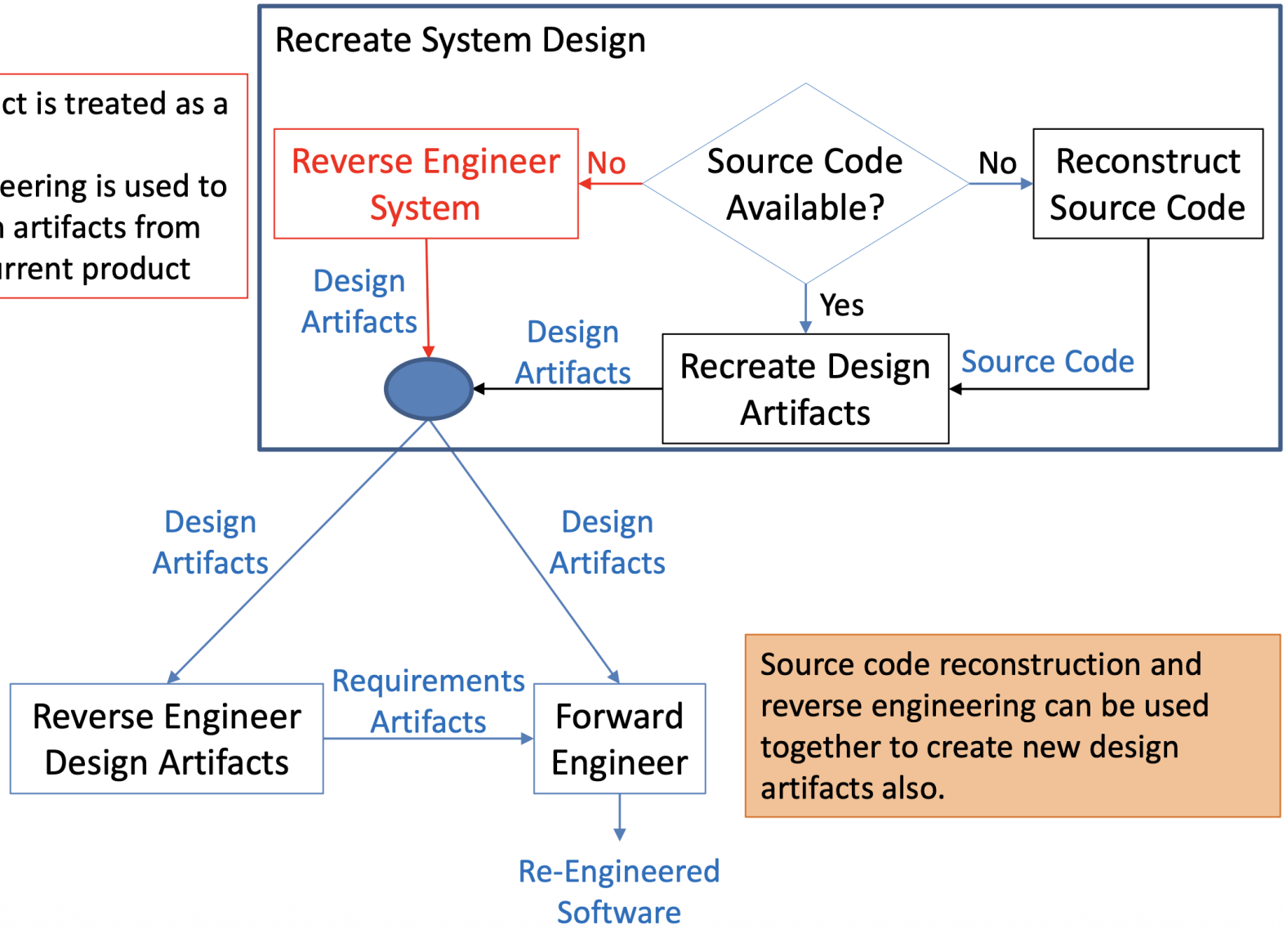


Problems with reverse compiler:

- Variable names are lost
- Compiler code optimization makes process difficult
- Constructs (e.g., a loop) in assembler correspond to different possible source code

Reverse Engineer System

- Existing product is treated as a black box
- Reverse engineering is used to deduce design artifacts from behavior of current product



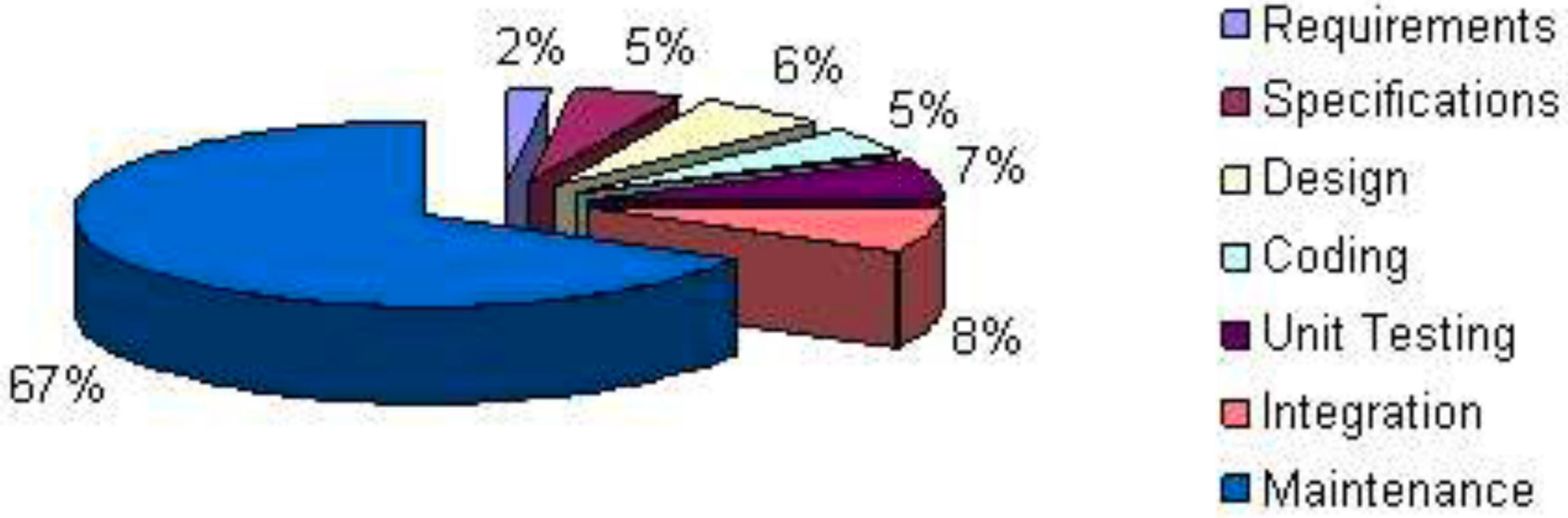
Source code reconstruction and reverse engineering can be used together to create new design artifacts also.

Software Construction Topics

- Greenfield & Brownfield
- Commenting
- Design by Contract (DbC)
- Open-Source Software Components
- Software Re-engineering
- **Software Maintenance**
- Refactoring

Software Life-Cycle Costs

- Software maintenance is 67% of the software life-cycle costs

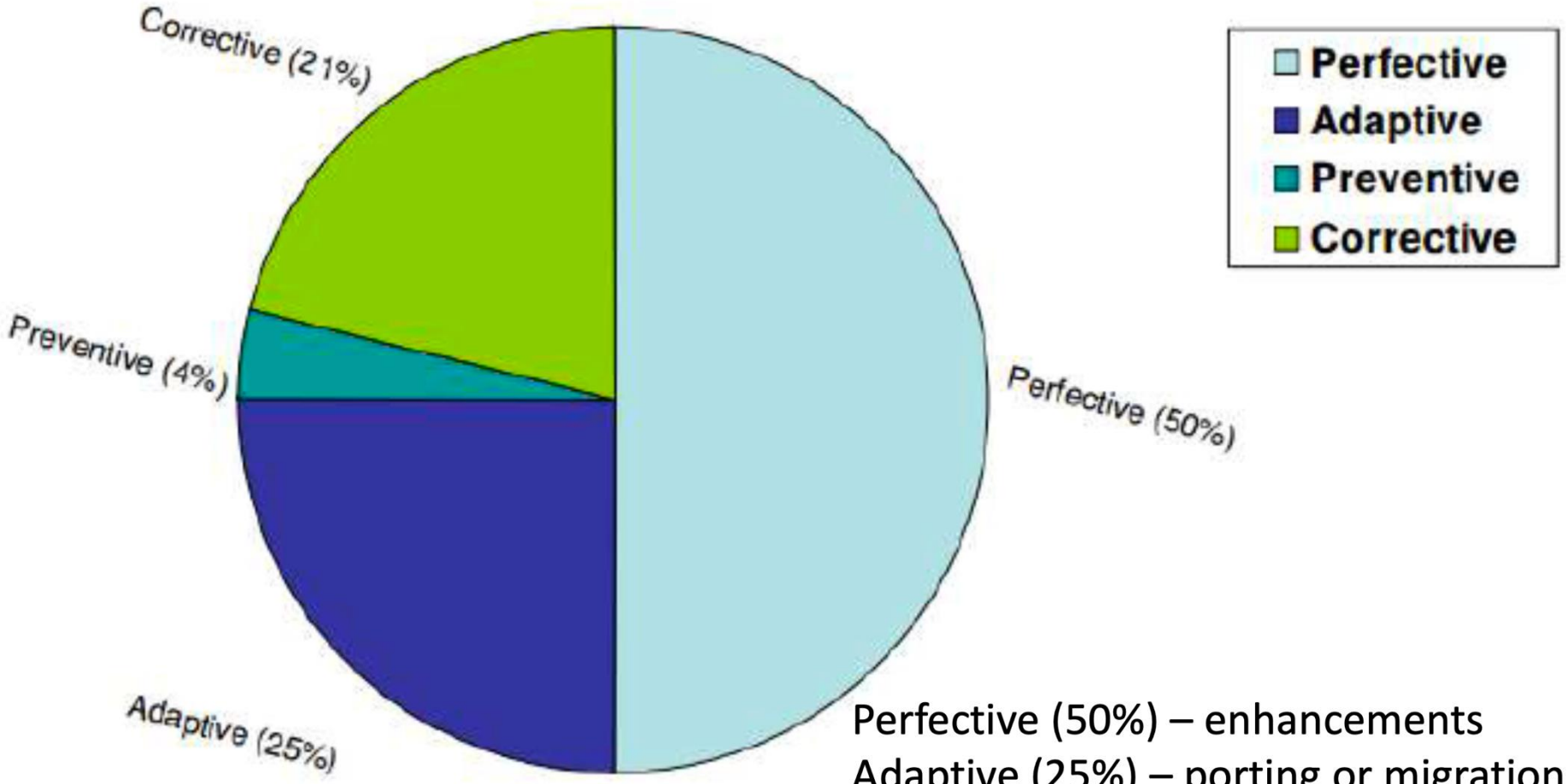


Software Maintenance

There are four different types of software maintenance:

- Perfective maintenance
  - Functional changes that are new or based on user requirements
  - Enhancements to increase the systems performance
- Adaptive maintenance
  - Environment change for the software.
  - Hardware, software, operating system, and organizational policy changes
- Corrective maintenance
  - Bugs, errors, faults in software that has already been released and is being used by the users.
- Preventive maintenance
  - Changes that ensure the longevity of the software
  - Code optimizing, code restructuring, and documentation.

Cost of Software Maintenance



Perfective (50%) – enhancements
Adaptive (25%) – porting or migration
Corrective (21%) – fix defects
Preventive (4%) - optimizing & restructuring

Perfective Maintenance (50%)

- Performed when you update the software system to improve its value, according to user demands.
- This includes:
  - Enhancement of software functionality
  - Improvement in user interfaces
  - Improvements in software usability
  - Speed optimization
  - Improvement in software performance
- It involves making enhancements in software functionality by implementing new or changed user requirements
- Changes are not considered a defect, error or fault.
- It is often, but not always, initiated by customer feedback.
- Example:
- Modifying an ERP (Enterprise Resource Planning) program to include a new payment settlement feature in a software system.

Adaptive Maintenance (25%)

- Updating and modifying the software when:
  - The platform in which your software operates is changing (due to technology, laws, policies, rules, operating system, etc.)
  - Your customers need the product to interface with new hardware or software.
  - You have predicted defects in the software that will affect your customers in the future.

Corrective Maintenance (21%)

- A software defect is an error, flaw, or fault in a computer program or system that causes it to produce an incorrect or unexpected result, or to behave in unintended ways.
- Software defects arises due to errors and faults in design, logic, and code of the software affecting users after you have released the software.
- A program that contains many defects, and/or defects that seriously interfere with its functionality, is said to be buggy (defective).
- Defects can trigger errors that may have ripple effects (e.g., memory leak)
- Defects may have subtle effects or cause the program to crash or freeze the computer.
- Other defects qualify as security defects and might, for example, enable a malicious user to bypass access controls in order to obtain unauthorized privileges.
- Corrective maintenance action (commonly referred to as “bug fixing”) addresses these errors and faults in your software system.

Corrective Maintenance (21%)

- Most commonly, defect reports are created by users and sent as feedback to the company that designed the software.
- Then the company's developers and testers review the code and make corrective changes to the software accordingly.
- If you detect and resolve the flaws in the software before users discover it, then the maintenance action is preventive or adaptive.
- However, if you fix the problem after getting defect reports from the user's side, then it is corrective maintenance action.

Corrective Maintenance (21%)

Essential features of defect tracking software:

- Reporting facility – complete with fields that will let you provide information about the defect, environment, module, severity, screenshots, etc.
- Assigning – Who is assigned to correcting the defect
- Tracking defects through the life cycle stages:
  - Those found in Verification of requirements, design, and coding artifacts
  - Those found during Validation
  - Those found by end-users during post-release maintenance
- History/work log/comments
- Reports – graphs or charts
- Storage and Retrieval – Every defect has an ID, which can be used to store, retrieve (search) and organize defect information.

Preventive Maintenance (4%)

- Preventive maintenance is a software change you make to prevent the occurrence of errors that might occur in the future.
- It increases the software maintainability by reducing its complexity.
- Preventive maintenance includes:

  – Updating the documentation: Updating the document according to the current state of the system.

  – Optimizing the code: Modifying the code for faster execution of programs or making efficient use of storage space.

  – Reconstructing the code: Transforming the structure of the program by reducing the source code, making it easily understandable, refactoring (more about this next time).

Question